ISLS Extension Phase 10 v1.0.0 (Revised v2)

Part A: C28 — Babylon Bridge
Part B: Benchmark Update B16–B24

Honest about what exists.
Honest about what must be built.
Honest about what must be measured.

Sebastian Klemm

18 March 2026

Abstract

Part A integrates the Babylon Compiler's IR layer (`glyph-ir`, `glyph-embed`, `glyph-canon`) as a structural validation and type-consistency backbone for the Forge. The Oracle (C25) still generates code — but now against an IR-derived structural contract. Codegen backends are **not** claimed to exist in Babylon or Merkaba; they are specified here as new code that must be written. Six backends (Rust, TypeScript, Python, SQL, YAML/Docker, Markdown) emit from IR structure + Oracle content.

Part B extends the benchmark suite from B01–B15 (core pipeline) to B16–B24, covering: Oracle synthesis latency, Oracle rejection rate, Pattern Memory hit rate, Foundry compilation success rate, Foundry average attempts, Template match accuracy, Studio endpoint latency, full forge-to-project throughput, and autonomy ratio over time.

Status: Implementation Specification.

New Crate: C28 (`isls-multilang`).

Depends on: C22, C23, C25, C26, C27; Babylon: `glyph-ir`, `glyph-embed`, `glyph-canon`, `glyph-q16`.

Contents

I Part A: C28 — Babylon Bridge 3

1 What Babylon Actually Provides 3

2 Integration Architecture 3

3 ArtifactIR to glyph-ir Conversion 4

4 Codegen Backends (Must Be Built) 5

5 Cross-Language Type Consistency 6

6 Embedding as Pre-Codegen Validation 7

7 Full-Stack Templates 7

8 CLI Extension 8

9 Acceptance Tests (Part A) 8

II Part B: Benchmark Update B16–B24 9

10 Current State 9

11 New Benchmarks B16–B24 9

12 Benchmark Implementation 11

13 Report Integration 12

14 Benchmark CLI 12

III Handoff 13

15 Dependency Map 13

16 Test Count 13

17 Completion Criteria 13

Part I
Part A: C28 — Babylon Bridge

1 What Babylon Actually Provides

An honest inventory of what exists in the Babylon Compiler repository and what does **not** exist:

Component	Exists?	Detail
`glyph-ir`	✓	IrDocument with 10 NodeKinds, 11 EdgeKinds, canonical sort, SHA-256 digest.
`glyph-embed`	✓	5-axis H5 embedding: structural coupling, functional density, topological complexity, symmetry, entropy. Q16 fixed-point.
`glyph-canon`	✓	JCS (RFC 8785) canonical JSON + SHA-256.
`glyph-q16`	✓	Q16.16 fixed-point arithmetic.
`glyph-gate`	✓	Obligation evaluation, tripolar gate, PoR FSM.
`glyph-mef`	✓	Append-only hash chain for compilation evidence.
`glyph-frontends`	✓	3 parsers (Sanskroot, HanLan, Cuneiform) lowering to glyph-ir.
Rust codegen backend	×	Does not exist.
TypeScript codegen backend	×	Does not exist.
Python codegen backend	×	Does not exist.
SQL codegen backend	×	Does not exist.
Any codegen backend	×	Babylon emits Crystal bundles, not source code in target languages.
Merkaba codegen backends	×	Mentioned in concepts but not implemented as functional emitters.

Remark 1.1. Babylon is a compiler that *reads* source code and produces certified knowledge artifacts. It does not *write* source code. The codegen backends must be written from scratch as part of C28.

2 Integration Architecture

The Honest Pipeline

```
DecisionSpec
  |
  v
[PMHD Drill] -> PmhdMonolith              (existing, C21)
  |
  v
[ArtifactIR] -> Components + Interfaces    (existing, C22)
  |
  v
[Bridge: ArtifactIR -> glyph-ir]           (NEW, C28)
  |   Converts components to IrNodes (Module, Function)
  |   Converts interfaces to IrEdges (CalleeRef, Contains)
  |   Deterministic, content-addressed
  v
[glyph-embed: compute H5 embedding]        (existing, Babylon)
```

```
  |   5 axes validate structural coherence
  |   Catches: disconnected modules, over-coupling,
  |   fragmentation, asymmetry, repetition
  v
[Oracle fills function bodies]            (existing, C25)
  |   Oracle receives: function name, signature, interfaces,
  |   constraints, H5 context
  |   Oracle returns: implementation code STRING for each function
  |   The Oracle does NOT return IR. It returns code.
  v
[Codegen Backend assembles files]         (NEW, C28)
  |   Takes: IR structure + Oracle body strings
  |   Produces: complete source files with correct
  |   imports, module structure, type definitions
  |   The backend writes the SCAFFOLDING.
  |   The Oracle writes the LOGIC.
  v
[Foundry: compile + test + fix loop]      (existing, C27)
  |
  v
[Crystal with IR digest + H5 embedding]
```

Remark 2.1 (Key Honesty). The Oracle still generates code strings. The codegen backends do **not** generate logic — they generate scaffolding (imports, module declarations, type signatures, function stubs, test scaffolding). The Oracle fills the function bodies. The backend assembles the final files from scaffolding + Oracle content.

This is a meaningful improvement over pure Oracle generation because: (a) imports are always correct (derived from IR edges), (b) type signatures are always consistent (derived from IR nodes), (c) module structure is always coherent (derived from IR topology), (d) only the *bodies* come from the Oracle, not the entire file.

3 ArtifactIR to glyph-ir Conversion

Definition 3.1 (Bridge Conversion).

```
pub fn artifact_to_glyph_ir(ir: &ArtifactIR) -> glyph_ir::IrDocument {
    let mut doc = glyph_ir::IrDocument::new(
        &ir.header.domain,
        &hex_encode(&ir.header.artifact_id),
    );

    // Root module
    let root = glyph_ir::IrNode::new("n_root", NodeKind::Module, "root")
        ;
    doc.nodes.push(root);

    // Each component -> Function node with metadata
    for (i, comp) in ir.components.iter().enumerate() {
        let id = format!("n_fn_{}", i);
        let mut node = glyph_ir::IrNode::new(&id, NodeKind::Function, &
            comp.name);
        node.params = if comp.dependencies.is_empty() {
            None
        } else {
            Some(comp.dependencies.clone())
```

```
        };
        // Store component content as property for Oracle prompt
        let mut props = BTreeMap::new();
        props.insert("kind".into(), json!(comp.kind));
        props.insert("description".into(), json!(comp.content));
        node.properties = Some(props);
        doc.nodes.push(node);
        doc.edges.push(glyph_ir::IrEdge::new("n_root", &id, EdgeKind::
            Contains));
    }

    // Interfaces -> edges
    for iface in &ir.interfaces {
        let provider_id = find_node_by_name(&doc, &iface.provider);
        let consumer_id = find_node_by_name(&doc, &iface.consumer);
        if let (Some(p), Some(c)) = (provider_id, consumer_id) {
            doc.edges.push(glyph_ir::IrEdge::new(&p, &c, EdgeKind::
                CalleeRef));
        }
    }

    doc.canonicalize();
    doc
}
```

4 Codegen Backends (Must Be Built)

Definition 4.1 (CodegenBackend Trait).

```
pub trait CodegenBackend: Send + Sync {
    fn language(&self) -> &str;
    fn extension(&self) -> &str;

    /// Generate scaffolding from IR structure.
    /// Returns files with stubs where Oracle content will be inserted.
    fn scaffold(&self, doc: &glyph_ir::IrDocument) -> Vec<ScaffoldFile>;

    /// Assemble final files by merging scaffolding with Oracle bodies.
    fn assemble(
        &self,
        scaffolds: &[ScaffoldFile],
        oracle_bodies: &BTreeMap<String, String>,  // node_id -> code
    ) -> Vec<EmittedFile>;
}

pub struct ScaffoldFile {
    pub path: String,
    pub language: String,
    pub imports: Vec<String>,          // derived from IR edges
    pub type_definitions: Vec<String>, // derived from IR nodes
    pub function_stubs: Vec<FunctionStub>,
}

pub struct FunctionStub {
    pub node_id: String,       // glyph-ir node ID
    pub name: String,
```

```
    pub signature: String,     // generated by backend from IR
    pub body_placeholder: String, // "// TODO: Oracle fills this"
}

pub struct EmittedFile {
    pub path: String,
    pub content: String,
    pub language: String,
    pub scaffold_lines: usize,  // lines from backend (structure)
    pub oracle_lines: usize,    // lines from Oracle (logic)
}
```

Definition 4.2 (Six Backends)**.**

Backend	Lang	Ext	Scaffolding It Generates
`RustBackend`	Rust	.rs	`mod` declarations, `pub struct` v `fn` signatures, `use` imports f Cargo.toml.
`TypeScriptBackend`	TS	.ts	`export interface` for types, `ex` signatures, `import` from IR edges
`PythonBackend`	Python	.py	`@dataclass class` for types, `def` type hints, `import` from IR ments.txt.
`SqlBackend`	SQL	.sql	`CREATE TABLE` from type nodes, from edges, migration numbering.
`OpsBackend`	YAML	.yml	Dockerfile skeleton, docker-comp pipeline steps. Structure-only, no
`DocsBackend`	Markdown	.md	README from IR root node, m Function nodes, API docs from Structure-only.

Key distinction: OpsBackend and DocsBackend are **fully deterministic** — they produce complete files from the IR alone, no Oracle needed. RustBackend, TypeScriptBackend, PythonBackend, SqlBackend produce scaffolding that needs Oracle bodies.

5 Cross-Language Type Consistency

Definition 5.1 (Type Node Convention)**.** A type is represented as an IR Function node with `kind = ‘‘type’’` in its properties and fields listed as parameters:

```
// IR representation of a "User" type:
IrNode {
    id: "n_type_user",
    kind: Function,  // glyph-ir's Function serves as container
    name: "User",
    params: Some(vec!["id:i64", "name:String", "email:String"]),
    properties: Some({"kind": "type", "description": "User entity"}),
}
```

Each backend interprets this IR node according to its language:

```
// RustBackend:
pub struct User { pub id: i64, pub name: String, pub email: String }

// TypeScriptBackend:
```

```
export interface User { id: number; name: string; email: string; }

// PythonBackend:
@dataclass
class User:
    id: int
    name: str
    email: str

// SqlBackend:
CREATE TABLE users (
    id BIGINT PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT NOT NULL
);
```

Consistency is structural: all backends read the same IR node. A field name mismatch is impossible because there is only one source.

6 Embedding as Pre-Codegen Validation

Definition 6.1 (Structural Pre-Check)**.** Before codegen, the H5 embedding is computed:

```
let embedding = glyph_embed::compute_embedding(&ir_doc);
```

Range checks per axis (configurable, defaults):

- a_1 (structural coupling): $[0.05, 0.80]$. Zero = disconnected. High = spaghetti.
- a_2 (functional density): $[0.10, 0.90]$. Zero = no functions. High = fragmented.
- a_3 (topological complexity): $[0.0, 0.70]$. High = over-nested.
- a_4 (symmetry): $[0.20, 1.0]$. Low = lopsided modules.
- a_5 (entropy): $[0.10, 0.90]$. Low = repetitive. High = incoherent.

If any axis is outside range, the system logs a warning and optionally re-decomposes (retry with different PMHD strategy). This is not a hard gate — it is diagnostic. The Foundry's compile/test loop is the hard gate.

7 Full-Stack Templates

Six new templates added to C26's catalog. Each is an IR tree (not a prompt template) with multi-language atoms:

ID	Name	Atoms	Mol.	Languages
T11	SaaS Starter	22	6	Rust + TS + SQL + Docker + MD
T12	Dashboard	18	5	Rust + TS + SQL + Docker
T13	API + Docs	12	3	Rust + MD + Docker
T14	Python ML	10	3	Python + SQL + Docker
T15	Static Site	8	2	HTML/CSS + MD
T16	Monorepo	16	5	Rust + TS + SQL + Docker + MD

Each template atom specifies its backend:

```
pub struct MultiLangAtom {
    pub name: String,
    pub backend: String,        // "rust", "typescript", "python", etc.
    pub fill: FillStrategy,     // Oracle, Static, Derive
```

```
    pub ir_node_ids: Vec<String>, // which IR nodes this atom covers
}
```

8 CLI Extension

```
# Forge with explicit target language
isls forge --spec intent.json --lang rust
isls forge --spec intent.json --lang typescript
isls forge --spec intent.json --lang python

# Forge full-stack (template auto-selects languages)
isls forge --spec intent.json --template saas-starter

# Emit IR for inspection
isls forge --spec intent.json --lang rust --dump-ir output.json

# Validate IR structure via H5 embedding
isls babylon check --ir output.json
```

9 Acceptance Tests (Part A)

ID	Name	Procedure
AT-BB1	IR conversion	Convert ArtifactIR to IrDocument; verify node count = component count + 1 (root).
AT-BB2	IR determinism	Convert same ArtifactIR twice; verify identical digest.
AT-BB3	H5 embedding	Compute embedding; verify all 5 axes in Q16 range.
AT-BB4	Rust scaffold	Generate scaffold from IR; verify .rs files with correct `pub fn` signatures.
AT-BB5	TS scaffold	Generate scaffold; verify .ts files with correct `export interface` definitions.
AT-BB6	Python scaffold	Generate scaffold; verify .py files with correct `@dataclass` definitions.
AT-BB7	SQL scaffold	Generate scaffold; verify .sql files with correct `CREATE TABLE`.
AT-BB8	Cross-lang types	Define type in IR; scaffold to Rust + TS; verify field names match exactly.
AT-BB9	Assembly	Scaffold + mock Oracle bodies; assemble; verify complete files with both scaffold and body content.

AT-BB10	Ops backend	Generate Dockerfile + compose from IR; verify valid YAML (no Oracle needed).
AT-BB11	Docs backend	Generate README from IR; verify contains module names and descriptions.
AT-BB12	Full-stack T11	Forge SaaS Starter; verify dirs for all 5 languages exist with correct files.

Part II
Part B: Benchmark Update B16–B24

10 Current State

The benchmark suite (C10 `isls-harness`) currently covers B01–B15:

ID	Name	Last Value	Phase
B01	Ingestion throughput	234,897 obs/sec	Core
B02	Graph update scaling	156 μs	Core
B03	Extraction scaling	103 ms	Core
B04	Cascade time	—	Core
B05	Dual consensus overhead	133%	Core
B06	Crystal serialization	55 μs	Core
B07	Replay speed	18,111 steps/sec	Core
B08	Evidence verification	5.0 μs	Core
B09	Memory scaling (5K ent.)	0.99 MB	Core
B10	Full macro-step	—	Core
B11	Registry resolution	0.07 μs	Phase 1
B12	Manifest construction	4 ms	Phase 1
B13	Manifest verification	3 ms	Phase 1
B14	Capsule roundtrip	10.1 μs	Phase 1
B15	Scheduler overhead	0%	Phase 1

Missing: Phases 5–9 (Forge, Oracle, Templates, Foundry, Studio) have zero benchmarks. The harness has no metrics for generative pipeline performance.

11 New Benchmarks B16–B24

ID	Name	Unit	What It Measures
B16	`forge_throughput`	crystals/sec	End-to-end: DecisionSpec → Crystal, averaged over 10 forge runs with test specs. Includes PMHD drill + ArtifactIR + Oracle mock + Forge pipeline.
B17	`oracle_latency`	ms/call	Average Oracle synthesis call latency. With mock Oracle (measures overhead, not API latency) and optionally with real API (measures end-to-end).
B18	`oracle_rejection_rate`	percent	Fraction of Oracle outputs that fail validation (parse, constraint, PMHD, gate). With mock Oracle: should be 0%. Tracked as M34.
B19	`pattern_hit_rate`	percent	Fraction of synthesis requests served from Pattern Memory without Oracle call. Initially 0%; grows over time. Tracked as M33 (autonomy ratio).
B20	`foundry_compile_rate`	percent	Fraction of Foundry fabrication runs where first attempt compiles. Measures Oracle code quality. In mock mode: 100% (mock always passes). In real mode: tracked per language.
B21	`foundry_avg_attempts`	attempts	Average number of compile-fix cycles per successful fabrication. Target: ≤ 2. Measures Oracle+Foundry loop efficiency.
B22	`template_match_accuracy`	percent	Fraction of test intents where auto-template-match selects the correct template. Measured against a labeled test set of 20 intents.
B23	`gateway_latency`	ms/request	Average round-trip for key Gateway endpoints: `GET /health`, `GET /crystals`, `POST /forge`.
B24	`full_fabrication_time`	seconds	Wall-clock time from DecisionSpec to a project on disk that passes `cargo test`. The ultimate end-to-end metric.

12 Benchmark Implementation

New benchmark functions in isls-harness

```
pub fn bench_forge_throughput(engine: &mut ForgeEngine) ->
   BenchmarkResult {
    let specs = generate_test_specs(10); // 10 diverse test specs
    let start = Instant::now();
    let mut crystals = 0;
    for spec in &specs {
        if engine.forge(spec.clone()).is_ok() {
            crystals += 1;
        }
    }
    let elapsed = start.elapsed();
    BenchmarkResult {
        id: "B16".into(),
        name: "forge_throughput".into(),
        value: crystals as f64 / elapsed.as_secs_f64(),
        unit: "crystals/sec".into(),
    }
}

pub fn bench_oracle_latency(oracle: &mut OracleEngine) ->
   BenchmarkResult {
    let test_prompts = generate_test_prompts(20);
    let start = Instant::now();
    for prompt in &test_prompts {
        let _ = oracle.synthesize_raw(prompt);
    }
    let elapsed = start.elapsed();
    BenchmarkResult {
        id: "B17".into(),
        name: "oracle_latency".into(),
        value: elapsed.as_millis() as f64 / 20.0,
        unit: "ms/call".into(),
    }
}

pub fn bench_template_match(catalog: &TemplateCatalog) ->
   BenchmarkResult {
    let test_set = labeled_intent_test_set(); // 20 (intent,
       expected_template) pairs
    let mut correct = 0;
    for (intent, expected) in &test_set {
        let matched = catalog.best_match_for_intent(intent);
        if matched.as_ref().map(|t| &t.name) == Some(expected) {
            correct += 1;
        }
    }
    BenchmarkResult {
        id: "B22".into(),
        name: "template_match_accuracy".into(),
        value: correct as f64 / test_set.len() as f64 * 100.0,
        unit: "percent".into(),
    }
```

```
}

pub fn bench_full_fabrication(foundry: &mut Foundry) -> BenchmarkResult
    {
    let spec = simple_rust_api_spec(); // "health check endpoint"
    let start = Instant::now();
    let result = foundry.fabricate(FoundryOperation::NewProject {
        spec,
        output_dir: temp_dir(),
    });
    let elapsed = start.elapsed();
    BenchmarkResult {
        id: "B24".into(),
        name: "full_fabrication_time".into(),
        value: elapsed.as_secs_f64(),
        unit: "seconds".into(),
    }
}
```

13 Report Integration

The HTML report gains a new section: **"11. Generative Pipeline Benchmarks (B16–B24)"** with the same layout as the existing benchmark table. The Studio Dashboard gains a "Forge Performance" panel showing B16, B21, B24 as headline numbers.

Report Section 11

```
<h2>11. Generative Pipeline Benchmarks</h2>
<table>
  <tr><td>B16</td><td>forge_throughput</td>
      <td>{value}</td><td>crystals/sec</td></tr>
  <tr><td>B17</td><td>oracle_latency</td>
      <td>{value}</td><td>ms/call</td></tr>
  <tr><td>B18</td><td>oracle_rejection_rate</td>
      <td>{value}</td><td>percent</td></tr>
  <tr><td>B19</td><td>pattern_hit_rate</td>
      <td>{value}</td><td>percent</td></tr>
  <tr><td>B20</td><td>foundry_compile_rate</td>
      <td>{value}</td><td>percent</td></tr>
  <tr><td>B21</td><td>foundry_avg_attempts</td>
      <td>{value}</td><td>attempts</td></tr>
  <tr><td>B22</td><td>template_match_accuracy</td>
      <td>{value}</td><td>percent</td></tr>
  <tr><td>B23</td><td>gateway_latency</td>
      <td>{value}</td><td>ms/request</td></tr>
  <tr><td>B24</td><td>full_fabrication_time</td>
      <td>{value}</td><td>seconds</td></tr>
</table>
```

14 Benchmark CLI

```
# Run all benchmarks (B01-B24)
isls bench
```

```
# Run only generative benchmarks (B16-B24)
isls bench --suite generative

# Run only core benchmarks (B01-B15)
isls bench --suite core

# Run a specific benchmark
isls bench --id B24

# Run with real Oracle (API key required)
isls bench --suite generative --oracle live
```

Part III
Handoff

15 Dependency Map

```
isls-multilang (C28)
  depends on:
    isls-types, isls-artifact-ir (C22), isls-forge (C23),
    isls-oracle (C25), isls-templates (C26), isls-foundry (C27)
  external:
    glyph-ir, glyph-embed, glyph-canon, glyph-q16
    (path deps to babylon-compiler workspace)

isls-harness (C10) -- MODIFIED
  new deps: isls-forge, isls-oracle, isls-templates,
            isls-foundry, isls-gateway
  new bench functions: B16-B24
```

16 Test Count

Phase	New Tests	Cumulative
Phase 1–9	326	326
Phase 10A (C28 Babylon Bridge)	12	338
Phase 10B (Benchmarks B16–B24)	9	$\geq$ 347

17 Completion Criteria

1. ArtifactIR converts to glyph-ir deterministically.
2. H5 embedding validates structural coherence before codegen.
3. Six codegen backends produce scaffolding from IR structure.
4. Oracle fills function bodies; backends assemble final files.
5. Cross-language type consistency from single IR node.
6. OpsBackend and DocsBackend produce complete files without Oracle.
7. Six full-stack templates (T11–T16) produce multi-language projects.

8. Babylon crates integrated as workspace dependencies.
9. B16–B24 benchmarks implemented and running.
10. HTML report includes Section 11 for generative benchmarks.
11. `isls bench --suite generative` works.
12. Total tests ≥ 347, zero warnings.
13. **Backends write scaffolding. Oracle writes logic. Benchmarks measure everything.**

Generated for ISLS Extension Phase 10 v1.0.0 (Revised v2).

Honest inventory. Honest architecture. Honest benchmarks.
Babylon validates structure. Backends write scaffolding.
Oracle writes logic. Foundry verifies. Benchmarks measure.
Nothing is claimed that does not exist.